



Persistent

Typescript Basics

Persistent Interactive | Persistent University

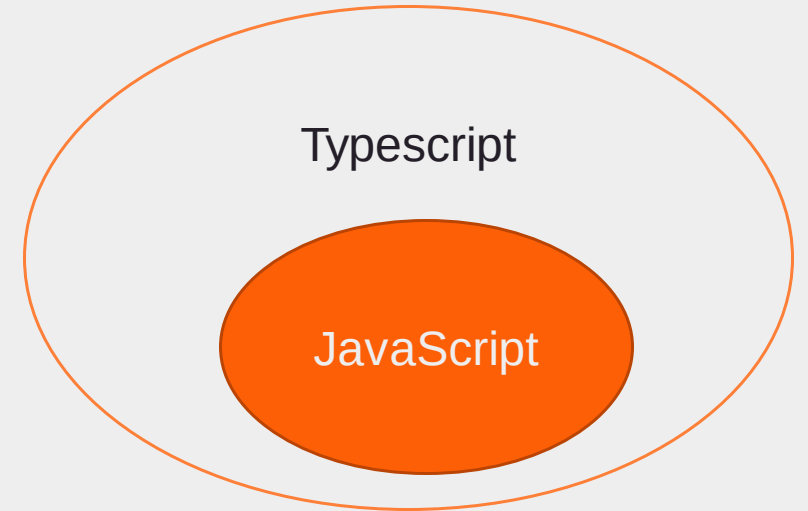


Key Learning Points

- Typescript vs JavaScript
- Datatypes supported
- Define Classes
- Arrow functions
- Import and Export

Typescript Overview

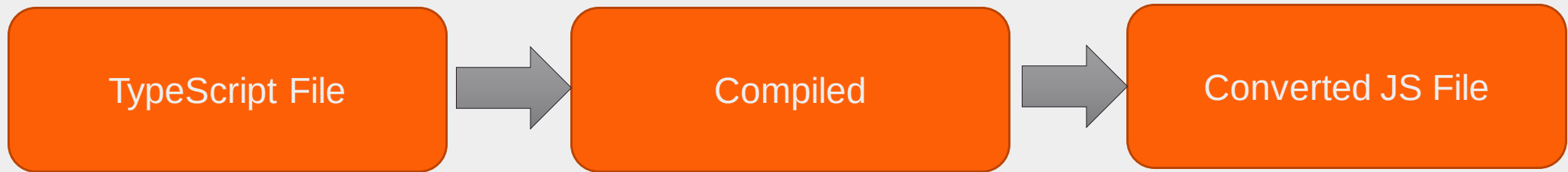
- Typed language : string, number, Boolean, enum
- Classes, interfaces, enums, access modifiers.
- Strict null checks
- Enhanced IDE support.
- More Object Oriented support as compared to JavaScript.



Transpiling Typescript to JavaScript

- **What scripting do browsers understand?**
- JavaScript files are included in html files and executed by the browser.
- So to take advantage of Typescript, there is a need to **transpile**.
- **Typescript code to be converted to JavaScript.**
- Even code written using features of ES2015 will still need to be transpiled since browsers don't yet understand them.
- Transpiling to Javascript ensures that code works in all browsers irrespective of the features used.

Transpilers



- **Transpilers** are tools that read source code written in one programming language, and produce the equivalent code in another language.

Typescript code

- Notice how the class is defined, use of constructor keyword and variable types used above as part of Typescript features.

```
export class User{
    private name:string;
    private age:number;
    private isIndian:boolean;
    constructor(name:string,age:number){
        this.name = name;
        this.age = age;
        this.isIndian = true;
    }
    getName():string{
        return this.name;
    }
}
```

Transpiled Javascript code

- Notice all types and typescript related code is removed when translating to JavaScript, but used by the IDE and compiler to spot errors.

```
"use strict";
exports.__esModule = true;
var User = /** @class */ (function () {
    function User(name, age) {
        this.name = name;
        this.age = age;
        this.isIndian = true;
    }
    User.prototype.getName = function () {
        return this.name;
    };
    return User;
}());
exports.User = User;
```

Getting started with Typescript

- The TypeScript compiler supports incremental compilation (--watch compiler flag), so that all subsequent changes can be compiled at greater speed.

```
npm install -g typescript
```

```
tsc User.ts
```

```
tsc -w User.ts
```


Basic Types

1. Boolean
2. Number
3. String
4. Array
5. Enum
6. Tuple
7. Any
8. Void

Declaring types variables

```
let isDone: boolean = false;
```

```
let age: number = 16;
```

```
let color: string = "blue";
```

```
let list: number[] = [1, 2, 3];
```

```
let list: Array<number> = [1, 2, 3];
```

```
enum Color {Red, Green, Blue}; let c: Color =  
    Color.Green;
```

More Types

- Tuple type
- Any type: Useful when data comes from user/third party.
- Compile time checks will be skipped here.

```
Let x: [string, number];  
X = ["hello", 10];
```

```
let x: any = 4;  
x = "Jane";  
x = false;
```

Some more Types

- Void type: More relevant to functions and not variables.

```
function display(): void { alert("Function  
execution ");  
}
```

- Template Strings with backtick (`) and embedded expressions

```
let fullName: string = `Tom Hamington`;
```

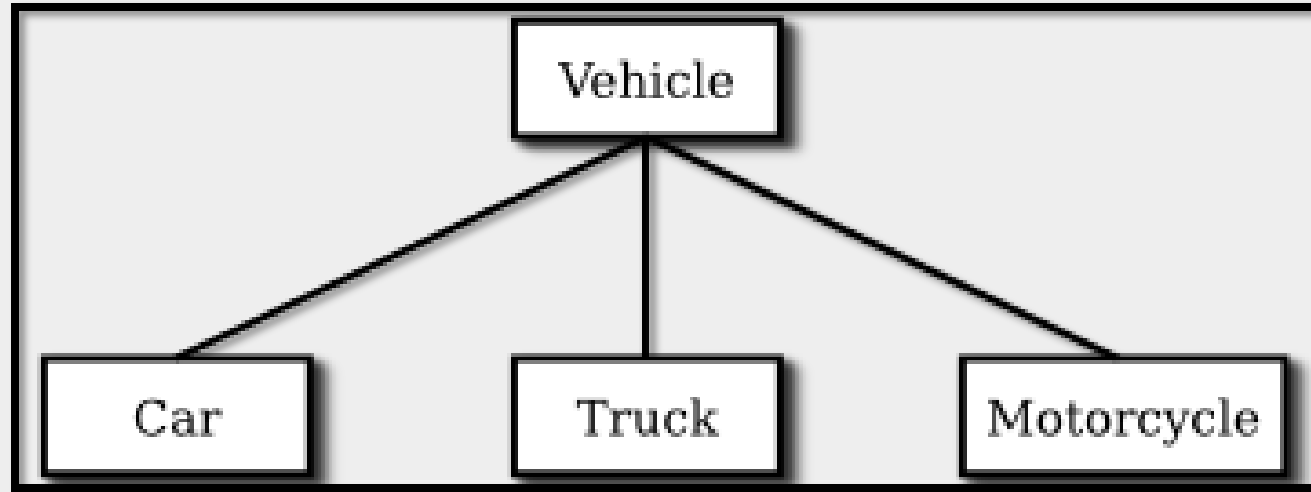
```
let msg: string = `Hello, my name is ${ fullName }`.
```

Classes

- Declaring a class in Typescript is more aligned to the approach used in other Object Oriented languages. Hence more convenient to use.
- Class has
 - properties,
 - functions
 - constructors

```
export class User{  
    private name:string;  
    private age:number;  
    private isIndian:boolean;  
    constructor(name:string,age:number){  
        this.name = name;  
        this.age = age;  
        this.isIndian = true;  
    }  
    getName():string{  
        return this.name;  
    }  
}
```

Implementation of Inheritance



- The **extends** keyword is used to create a subclass.
- Subclasses can override the methods from the base class.
- Derived classes that contain constructor functions must call `super()` which will execute the constructor function on the base class.

Modifiers

- Public by default
- Private
- Protected

Typed Functions

- We can add types to each of the parameters and then to the function itself to add a return type.

```
function add(x: number, y: number): number {  
    return x + y;  
}
```


Exporting and Importing

- Starting with the ECMAScript 2015, JavaScript has a concept of modules extended to Typescript as well.
- Classes, Interfaces , variables, functions can be exported and imported.

```
export const maxCount = 100;  
export class ZipCodeValidator {  
    isAcceptable(s: string) {  
        return s.length === 5 ;  
    }  
}
```

Lambda Expressions in Typescript (Fat Arrow)

```
var findSum = function(num1,num2){  
    return num1+num2;  
}
```



```
var findSum = (num1,num2) → {  
    return num1+num2;  
}
```

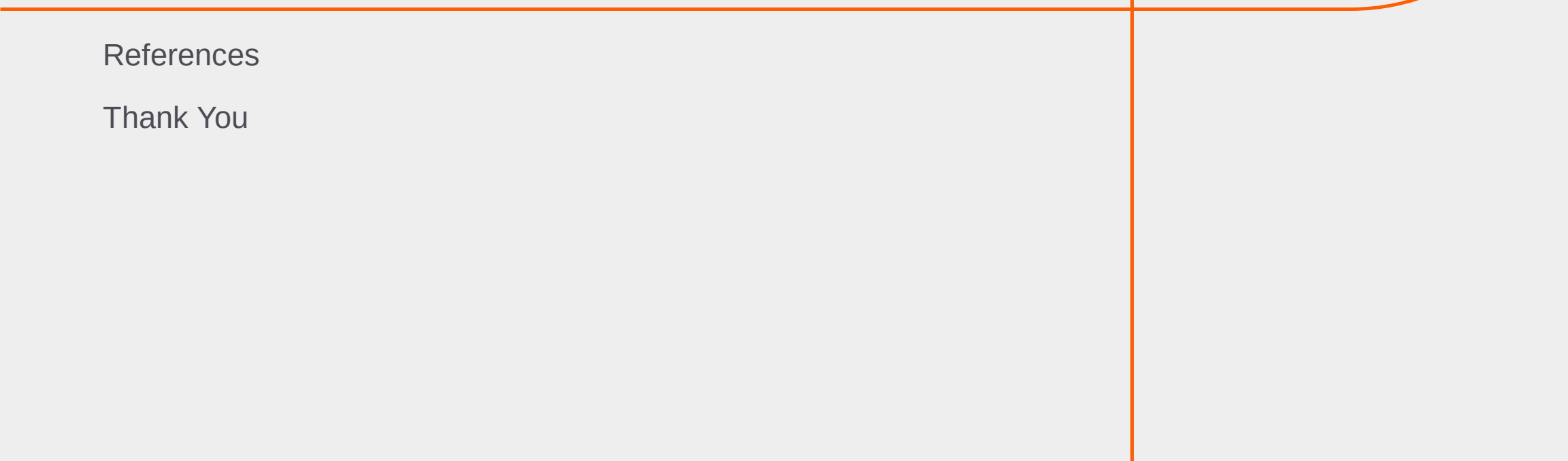


```
var findSum = (num1,num2) → return num1+num2;
```

Summary: Session

- Installing Typescript
- Transpiling Typescript to JavaScript
- Writing Classes in Typescript
- Commonly used Data types
- Exporting and Importing Classes

Appendix



References

Thank You

References

- <https://www.typescriptlang.org/>
- <https://www.tutorialsteacher.com/typescript>

Key Contacts :

Persistent University :

- Archana Ghatigar
archana_ghatigar@persistent.co.in



Thank you!

Persistent Interactive | Persistent University

